

■ Détection de Fraude sur Carte Bancaire

Credit Card Fraud Detection · Machine Learning & Deep Learning · Kaggle Dataset

Python 3.x

Scikit-learn

XGBoost

TensorFlow/Keras

Pandas · NumPy

■ Résumé du Projet

Ce projet s'attaque à l'un des défis les plus critiques de la fintech moderne : la **détection automatique de transactions frauduleuses** sur carte bancaire. Sur un dataset de **284 807 transactions** réelles (anonymisées via PCA), nous comparons quatre approches algorithmiques pour identifier les 492 fraudes (**0.17 % du total**) avec la meilleure combinaison précision/rappel possible.

Chiffres clés

284 807

Transactions analysées

492

Fraudes réelles

0.17 %

Taux de fraude

84 %

Meilleur F1 (fraude)

4

Modèles comparés

■ À Propos du Dataset

Source : Kaggle — Credit Card Fraud Detection (ULB)

- **284 807 lignes** · transactions européennes sur 2 jours (septembre 2013)
- **31 colonnes** : Time, Amount, V1–V28 (composantes PCA anonymisées), Class
- **Aucune valeur manquante** — dataset propre mais fortement déséquilibré
- **Variable cible** : Class (0 = normale, 1 = fraude)
- Les variables V1–V28 sont des **composantes principales issues d'une ACP** — les noms originaux sont confidentiels

■■ Pipeline Méthodologique

01 Chargement & Exploration

284 807 transactions · 31 colonnes · PCA anonymisée

02 Gestion des Outliers

Transformation par racine cubique (cbrt) — évite log sur valeurs négatives

03 Analyse du déséquilibre

0.17 % de fraudes · Stratégie : `class_weight` & métriques adaptées

04 Prétraitement & Split

70/30 stratifié · `StandardScaler` sur features V1–V28, Amount, Time

05 Modélisation

Logistic Reg · Random Forest · XGBoost · Deep Learning (MLP 4 couches)

06 Évaluation

Precision / Recall / F1 · Confusion matrix · Learning curves · Feature importance

■ Défis Techniques & Décisions Clés

■ Déséquilibre de classes extrême (99.83 % vs 0.17 %)

SMOTE et sur-échantillonnage rejetés (risque d'overfitting avec un ratio de 577:1). Solution retenue : **`class_weight='balanced'`** pour Random Forest, **`scale_pos_weight`** pour XGBoost, et focus sur F1/Recall comme métriques.

■ Outliers massifs sur presque toutes les variables

Log-transformation impossible (valeurs négatives présentes). Capping IQR trop agressif. Solution retenue : **transformation racine cubique** (`np.cbrt`) — robuste aux valeurs négatives, réduit considérablement les outliers.

■ Absence de noms de variables (anonymisation PCA)

Impossible de faire de la sélection de features basée sur la connaissance métier. Analyse par **importance XGBoost** — V14 ressort comme variable la plus discriminante, ce qui est cohérent avec la littérature sur ce dataset.

■ Choix des métriques d'évaluation

L'accuracy est trompeuse (99.83 % en prédisant toujours 'normale'). Focus sur **Precision, Recall et F1-Score de la classe 1** exclusivement. La matrice de confusion permet de quantifier faux positifs vs faux négatifs.

■ Résultats & Comparaison des Modèles

Modèle	Précision (Fraude)	Rappel (Fraude)	F1-Score (Fraude)	Statut
Logistic Regression	5 %	87 %	9 %	■ Rejeté
Random Forest	97 %	70 %	82 %	■ Retenu
XGBoost	93 %	76 %	84 %	■ Retenu
Deep Learning (MLP)	80 %	78 %	79 %	■■ Overfitting

■ **Modèle retenu : XGBoost** — meilleur équilibre précision/rappel (F1 = 84 %). Random Forest offre une précision plus haute (97 %) mais un rappel plus faible (70 %). La Régression Logistique, bien qu'ayant un bon rappel (87 %), génère trop de faux positifs (précision 5 %) — inacceptable en production bancaire. Le Deep Learning montre de l'overfitting dès le début de l'entraînement.

■ Feature Importance (XGBoost)

L'analyse de l'importance des features via XGBoost révèle que la variable **V14** est de loin la plus discriminante pour détecter les fraudes, suivie par V4, V12 et V10. Ce résultat est cohérent avec la littérature scientifique sur ce dataset (souvent associé à des patterns de comportement post-transaction). La variable **Time** contribue peu au modèle.

■ Structure du Repository

```
defi-data-science/  
■■■ notebooks/  
■ ■■■ creditcard_fraud_detection.ipynb  
■■■ data/  
■ ■■■ creditcard.csv (télécharger via Kaggle)  
■■■ README.md  
■■■ requirements.txt
```

■ Lancer le Projet

```
# 1. Cloner le repository  
git clone https://github.com/<username>/defi-data-science  
  
# 2. Installer les dépendances  
pip install -r requirements.txt  
  
# 3. Télécharger le dataset Kaggle  
kaggle datasets download mlg-ulb/creditcardfraud  
  
# 4. Lancer Jupyter
```

```
jupyter notebook notebooks/creditcard_fraud_detection.ipynb
```

```
pandas>=1.5 · numpy>=1.23 · scikit-learn>=1.2  
xgboost>=1.7 · tensorflow>=2.12 · matplotlib>=3.6 · seaborn>=0.12
```

■ Pistes d'Amélioration

- **SHAP values** — interprétabilité granulaire par transaction (pourquoi ce score ?)
- **Seuil de décision adaptatif** — optimiser le seuil Precision/Recall selon le coût métier
- **Isolation Forest / Autoencoder** — approches non-supervisées de détection d'anomalies
- **Déploiement API** — FastAPI + Docker pour servir le modèle en temps réel
- **Monitoring en production** — drift detection sur les distributions V1–V28